- If you don’t limit the time your client waits, you risk your computational resources being blocked for too long (for imperative applications). In the worst-case scenario, your application will be completely unresponsive because all the available threads are blocked, waiting for responses from a remote service, and there are no threads available to handle new requests.
- If you can’t meet your Service Level Agreements (SLAs), there’s no reason to keep waiting for an answer. It’s better to fail the request.

Here are some examples of timeouts:

- *Connection timeout*—This is the time limit for establishing a communication channel with a remote resource. Earlier you configured the `server.netty.connection-timeout` property to limit the time Netty waits for a TCP connection to be established.
- *Connection pool timeout*—This is the time limit for a client to get a connection from a pool. In chapter 5, you configured a timeout for the Hikari connection pool through the `spring.datasource.hikari.connection-timeout` property.
- *Read timeout*—This is the time limit for reading from a remote resource after establishing the initial connection. In the following sections, you’ll define a read timeout for the call to the Catalog Service performed by the `BookClient` class.

In this section, you’ll define a timeout for `BookClient` so that if it expires, the Order Service application will throw an exception. You can also